ENSF 519 Assignment 1

Wardrobe.io

Selenium WebDriver Test Cases

Alessandro Baldassarre

Group: L02-14

October 16, 2026

Overview

Wardrobe.io is a web app where users keep a digital wardrobe: they add, view and delete clothing items, build outfits, and list clothes for sale. This document describes the automated end-to-end tests written for the site using Python, Selenium WebDriver and pytest. Each test drives a real Chrome browser against the running app (frontend at http://localhost:5173, backend API at http://localhost:3000).

By default each run registers a fresh throwaway test account (an existing account can be used instead by setting TEST_USER_EMAIL and TEST_USER_PASSWORD in tests/.env). Any clothing items a test creates are deleted afterwards, so the tests are repeatable.

Running the tests

Start the app first (database and backend with Docker, frontend with Vite), then run the tests from the tests folder:

```
docker compose up -d              # from the project root: database + backend
cd frontend && npm run dev        # frontend on http://localhost:5173

cd tests
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt   # first time only
.venv/bin/pytest                  # run every test in a visible Chrome window
.venv/bin/pytest --headless       # run without opening a window
.venv/bin/pytest test_auth.py     # run one file
.venv/bin/pytest --slow-mo=1      # pause 1 second after each action (for demos)
```

Test case summary

ID	Test case	Script
TC-01	Successful login	`test_auth.py::test_login_success`
TC-02	Login with wrong password	`test_auth.py::test_login_wrong_password`
TC-03	Login with empty fields	`test_auth.py::test_login_empty_fields`
TC-04	Register a new account	`test_auth.py::test_register_new_user`
TC-05	Register with an email already in use	`test_auth.py::test_register_existing_email`
TC-06	Logout	`test_auth.py::test_logout`
TC-07	Protected page while logged out	`test_auth.py::test_protected_page_redirects_when_logged_out`
TC-08	Add a clothing item	`test_wardrobe.py::test_add_item_appears_in_wardrobe`
TC-09	Added item persists after refresh	`test_wardrobe.py::test_added_item_persists_after_refresh`
TC-10	Add item with required fields missing	`test_wardrobe.py::test_add_item_missing_required_fields`
TC-11	Add item without an image	`test_wardrobe.py::test_add_item_without_image_shows_alert`
TC-12	Delete a clothing item	`test_wardrobe.py::test_delete_item`
TC-13	Navigation links load the correct page	`test_navigation.py::test_nav_link_loads_page`

TC-01

Successful login

`tests/test_auth.py::test_login_success`

Description	Verifies that a registered user can log in with valid credentials and is taken into the protected part of the site.
Steps	1. Open the login page (/login). 2. Enter the test user's email and password. 3. Click the "Log in" button.
Desired output	The browser is redirected to /wardrobe, and the "Logout" button and "Wardrobe" nav link (only shown to logged-in users) are visible.

Python script (Selenium WebDriver)

```
def test_login_success(driver, wait, open_page, test_user):
    open_page("/login")
    helpers.submit_login(driver, wait, test_user["email"], test_user["password"])

    wait.until(EC.url_contains("/wardrobe"))
    # Only logged-in users see the Logout button and the nav links.
    assert wait.until(EC.visibility_of_element_located(helpers.LOGOUT_BUTTON))
    assert driver.find_element(By.LINK_TEXT, "Wardrobe").is_displayed()
```

Shared fixtures (`driver`, `wait`, `logged_in_driver`, ...) are defined in conftest.py and the `helpers` functions in helpers.py; both are reproduced in full in the appendix.

TC-02

Login with wrong password

`tests/test_auth.py::test_login_wrong_password`

Description	Verifies that logging in with a correct email but an incorrect password is rejected.
Steps	1. Open the login page (/login). 2. Enter the test user's email and a wrong password. 3. Click the "Log in" button.
Desired output	The error message "Invalid credentials" is displayed and the browser stays on /login.

Python script (Selenium WebDriver)

```
def test_login_wrong_password(driver, wait, open_page, test_user):
    open_page("/login")
    helpers.submit_login(driver, wait, test_user["email"], "definitely-wrong-password")

    error = wait.until(EC.visibility_of_element_located(helpers.LOGIN_ERROR))
    assert error.text == "Invalid credentials"
    assert driver.current_url.endswith("/login")
```

Shared fixtures (`driver`, `wait`, `logged_in_driver`, ...) are defined in conftest.py and the `helpers` functions in helpers.py; both are reproduced in full in the appendix.

TC-03

Login with empty fields

`tests/test_auth.py::test_login_empty_fields`

Description	Verifies that submitting the login form with no email or password does not log the user in.
Steps	1. Open the login page (/login). 2. Leave the email and password fields empty. 3. Click the "Log in" button.
Desired output	An error message is displayed and the browser stays on /login.

Python script (Selenium WebDriver)

```
def test_login_empty_fields(driver, wait, open_page):
    # The login inputs aren't marked `required`, so the browser lets the form
    # submit and the backend rejects it.
    open_page("/login")
    wait.until(EC.element_to_be_clickable(helpers.LOGIN_SUBMIT)).click()

    error = wait.until(EC.visibility_of_element_located(helpers.LOGIN_ERROR))
    assert error.text != ""
    assert driver.current_url.endswith("/login")
```

Shared fixtures (`driver`, `wait`, `logged_in_driver`, ...) are defined in conftest.py and the `helpers` functions in helpers.py; both are reproduced in full in the appendix.

TC-04

Register a new account

`tests/test_auth.py::test_register_new_user`

Description	Verifies that a new visitor can create an account and then log in with it.
Steps	1. Open the registration page (/register). 2. Enter a first name, last name, unique username, unique email (made unique with a timestamp) and a password. 3. Click the "Create Account" button. 4. On the login page, enter the new email and password and click "Log in".
Desired output	After registering, the browser is redirected to /login. Logging in with the new account redirects to /wardrobe and shows the "Logout" button.

Python script (Selenium WebDriver)

```
def test_register_new_user(driver, wait, open_page):
    stamp = int(time.time() * 1000)
    email = f"register_{stamp}@example.com"
    password = "Password123!"

    open_page("/register")
    _fill_register_form(driver, wait, email, f"register_{stamp}", password)

    # A successful sign-up sends the user to the login page...
    wait.until(EC.url_contains("/login"))

    # ...and the new account can actually log in.
    helpers.submit_login(driver, wait, email, password)
    wait.until(EC.url_contains("/wardrobe"))
    assert wait.until(EC.visibility_of_element_located(helpers.LOGOUT_BUTTON))
```

Shared fixtures (`driver`, `wait`, `logged_in_driver`, ...) are defined in conftest.py and the `helpers` functions in helpers.py; both are reproduced in full in the appendix.

TC-05

Register with an email already in use

`tests/test_auth.py::test_register_existing_email`

Description	Verifies that an account cannot be registered with an email address that is already in use.
Steps	1. Open the registration page (/register). 2. Enter a first name, last name, a new unique username, the email of an existing account, and a password. 3. Click the "Create Account" button.
Desired output	The error message "Email or username already in use" is displayed and the browser stays on /register.

Python script (Selenium WebDriver)

```
def test_register_existing_email(driver, wait, open_page, test_user):
    open_page("/register")
    # Unique username, so the email is the only thing that clashes.
    _fill_register_form(
        driver,
        wait,
        test_user["email"],
        f"dupe_{int(time.time() * 1000)}",
        "Password123!",
    )

    error = wait.until(
        EC.visibility_of_element_located((By.XPATH, "//form/p[contains(., 'already in use')]"))
    )
    assert "Email or username already in use" in error.text
    assert driver.current_url.endswith("/register")
```

Shared fixtures (`driver`, `wait`, `logged_in_driver`, ...) are defined in conftest.py and the `helpers` functions in helpers.py; both are reproduced in full in the appendix.

TC-06

Logout

`tests/test_auth.py::test_logout`

Description	Verifies that logging out ends the user's session.
Steps	1. Log in as the test user (lands on /wardrobe). 2. Click the "Logout" button in the header. 3. Navigate directly to /wardrobe.
Desired output	After clicking "Logout" the browser is redirected to /login, and visiting /wardrobe afterwards redirects to /login again instead of showing the wardrobe.

Python script (Selenium WebDriver)

```
def test_logout(logged_in_driver, wait, open_page):
    driver = logged_in_driver
    driver.find_element(*helpers.LOGOUT_BUTTON).click()

    # Logging out on a protected page bounces the user to /login.
    wait.until(EC.url_contains("/login"))

    # And the session is really gone: /wardrobe redirects again.
    open_page("/wardrobe")
    wait.until(EC.url_contains("/login"))
    assert "/wardrobe" not in driver.current_url
```

Shared fixtures (`driver`, `wait`, `logged_in_driver`, ...) are defined in conftest.py and the `helpers` functions in helpers.py; both are reproduced in full in the appendix.

TC-07

Protected page while logged out

`tests/test_auth.py::test_protected_page_redirects_when_logged_out`

Description	Verifies that protected pages cannot be viewed without logging in.
Steps	1. Without logging in, navigate directly to /wardrobe.
Desired output	The browser is redirected to /login.

Python script (Selenium WebDriver)

```
def test_protected_page_redirects_when_logged_out(driver, wait, open_page):
    open_page("/wardrobe")

    wait.until(EC.url_contains("/login"))
    assert driver.current_url.endswith("/login")
```

Shared fixtures (`driver`, `wait`, `logged_in_driver`, ...) are defined in conftest.py and the `helpers` functions in helpers.py; both are reproduced in full in the appendix.

TC-08

Add a clothing item

`tests/test_wardrobe.py::test_add_item_appears_in_wardrobe`

Description	Verifies that a user can add a clothing item and that it is saved with the details they entered.
Steps	1. Log in as the test user (lands on /wardrobe). 2. Click "Add" to open the "Add Clothing Item" form. 3. Enter a unique name, brand "Uniqlo", type "Tops", size "L", color "Navy", and choose an image file. 4. Click "Add Item". 5. Click the new item's card to open its detail page.
Desired output	The new item's card appears in the wardrobe showing "Uniqlo | Size L". Its detail page shows the item name as the heading and lists "Brand: Uniqlo", "Type: Tops", "Color: Navy" and "Size: L".

Python script (Selenium WebDriver)

```
def test_add_item_appears_in_wardrobe(
    logged_in_driver, wait, test_image, unique_name, cleanup_items
):
    driver = logged_in_driver
    cleanup_items.append(unique_name)

    card = helpers.add_item(
        driver,
        wait,
        unique_name,
        test_image,
        brand="Uniqlo",
        item_type="Tops",
        size="L",
        color="Navy",
    )

    # The card in the grid shows the brand and size...
    summary = card.find_element(By.XPATH, "following-sibling::p")
    assert summary.text == "Uniqlo | Size L"

    # ...and the detail page shows everything that was entered.
    card.click()
    wait.until(EC.visibility_of_element_located(helpers.item_heading(unique_name)))
    details = driver.find_element(By.TAG_NAME, "main").text
    assert "Brand: Uniqlo" in details
    assert "Type: Tops" in details
    assert "Color: Navy" in details
    assert "Size: L" in details
```

Shared fixtures (`driver`, `wait`, `logged_in_driver`, ...) are defined in conftest.py and the `helpers` functions in helpers.py; both are reproduced in full in the appendix.

TC-09

Added item persists after refresh

`tests/test_wardrobe.py::test_added_item_persists_after_refresh`

Description	Verifies that added clothing items are stored in the database, not just shown on the page.
Steps	1. Log in as the test user (lands on /wardrobe). 2. Add a clothing item with a unique name, type, size, color and image. 3. Reload the page in the browser.
Desired output	After the reload, the new item's card is still displayed in the wardrobe.

Python script (Selenium WebDriver)

```
def test_added_item_persists_after_refresh(
    logged_in_driver, wait, test_image, unique_name, cleanup_items
):
    driver = logged_in_driver
    cleanup_items.append(unique_name)
    helpers.add_item(driver, wait, unique_name, test_image)

    driver.refresh()

    assert wait.until(EC.visibility_of_element_located(helpers.item_card_link(unique_name)))
```

Shared fixtures (`driver`, `wait`, `logged_in_driver`, ...) are defined in conftest.py and the `helpers` functions in helpers.py; both are reproduced in full in the appendix.

TC-10

Add item with required fields missing

`tests/test_wardrobe.py::test_add_item_missing_required_fields`

Description	Verifies that the add item form cannot be submitted when required fields are empty.
Steps	1. Log in as the test user (lands on /wardrobe). 2. Click "Add" to open the "Add Clothing Item" form. 3. Leave every field empty and click "Add Item".
Desired output	The browser's required-field validation stops the submission (the Name field is flagged as missing with a validation message), the form stays open, and no item is saved to the wardrobe.

Python script (Selenium WebDriver)

```
def test_add_item_missing_required_fields(logged_in_driver, wait, unique_name):
    driver = logged_in_driver
    helpers.open_add_item_form(driver, wait)

    # Submit with nothing filled in; the browser's `required` check blocks it.
    driver.find_element(*helpers.ADD_ITEM_SUBMIT).click()

    name_input = driver.find_element(*helpers.NAME_INPUT)
    assert driver.execute_script("return arguments[0].validity.valueMissing", name_input)
    assert name_input.get_attribute("validationMessage") != ""
    # The form is still open and nothing was saved.
    assert driver.find_element(*helpers.ADD_ITEM_FORM_TITLE).is_displayed()
    assert unique_name not in _wardrobe_names(driver)
```

Shared fixtures (`driver`, `wait`, `logged_in_driver`, ...) are defined in conftest.py and the `helpers` functions in helpers.py; both are reproduced in full in the appendix.

TC-11

Add item without an image

`tests/test_wardrobe.py::test_add_item_without_image_shows_alert`

Description	Verifies that a clothing item cannot be added without an image.
Steps	1. Log in as the test user (lands on /wardrobe). 2. Click "Add" to open the "Add Clothing Item" form. 3. Enter a unique name, type "Shoes", size "9" and color "White", but do not choose an image. 4. Click "Add Item".
Desired output	A browser alert appears with the message "Please select an image", and no item is saved to the wardrobe.

Python script (Selenium WebDriver)

```
def test_add_item_without_image_shows_alert(logged_in_driver, wait, unique_name):
    driver = logged_in_driver
    helpers.open_add_item_form(driver, wait)
    helpers.fill_add_item_form(
        driver, name=unique_name, item_type="Shoes", size="9", color="White"
    )

    driver.find_element(*helpers.ADD_ITEM_SUBMIT).click()

    alert = wait.until(EC.alert_is_present())
    assert alert.text == "Please select an image"
    alert.accept()
    assert unique_name not in _wardrobe_names(driver)
```

Shared fixtures (`driver`, `wait`, `logged_in_driver`, ...) are defined in conftest.py and the `helpers` functions in helpers.py; both are reproduced in full in the appendix.

TC-12

Delete a clothing item

`tests/test_wardrobe.py::test_delete_item`

Description	Verifies that a user can delete a clothing item from their wardrobe.
Steps	1. Log in as the test user (lands on /wardrobe). 2. Add a clothing item with a unique name. 3. Click the item's card to open its detail page. 4. Click the "Delete" button.
Desired output	The browser is redirected back to /wardrobe, the item's card is no longer displayed, and the item no longer exists in the user's saved wardrobe.

Python script (Selenium WebDriver)

```
def test_delete_item(logged_in_driver, wait, test_image, unique_name, cleanup_items):
    driver = logged_in_driver
    cleanup_items.append(unique_name)
    helpers.add_item(driver, wait, unique_name, test_image).click()

    wait.until(EC.visibility_of_element_located(helpers.item_heading(unique_name)))
    driver.find_element(*helpers.button_with_text("Delete")).click()

    # Deleting sends the user back to the wardrobe without the item.
    wait.until(EC.url_matches(r"/wardrobe$"))
    assert unique_name not in _wardrobe_names(driver)
    assert driver.find_elements(*helpers.item_card_link(unique_name)) == []
```

Shared fixtures (`driver`, `wait`, `logged_in_driver`, ...) are defined in conftest.py and the `helpers` functions in helpers.py; both are reproduced in full in the appendix.

TC-13

Navigation links load the correct page

`tests/test_navigation.py::test_nav_link_loads_page`

Description	Verifies that each link in the navigation bar opens the correct page. The test runs once for each link: Wardrobe, Listings, Outfits and Profile.
Steps	1. Log in as the test user (lands on /wardrobe). 2. Click the nav link being tested.
Desired output	• Wardrobe opens /wardrobe with the heading "All Clothing". • Listings opens /listings with the heading "All Clothing". • Outfits opens /outfits with the heading "Outfits". • Profile opens /profile/favourites with the heading "Favourites".

Python script (Selenium WebDriver)

```
@pytest.mark.smoke
@pytest.mark.parametrize(
    "link_text, expected_path, heading",
    [
        ("Wardrobe", "/wardrobe", "//main//h2[normalize-space()='All Clothing']"),
        ("Listings", "/listings", "//main//h2[normalize-space()='All Clothing']"),
        ("Outfits", "/outfits", "//main//h1[normalize-space()='Outfits']"),
        ("Profile", "/profile/favourites", "//main//h2[normalize-space()='Favourites']"),
    ],
    ids=["wardrobe", "listings", "outfits", "profile"],
)
def test_nav_link_loads_page(logged_in_driver, wait, link_text, expected_path, heading):
    driver = logged_in_driver

    wait.until(EC.element_to_be_clickable((By.LINK_TEXT, link_text))).click()

    wait.until(EC.url_matches(f"{expected_path}$"))
    assert wait.until(EC.visibility_of_element_located((By.XPATH, heading)))
```

Shared fixtures (`driver`, `wait`, `logged_in_driver`, ...) are defined in conftest.py and the `helpers` functions in helpers.py; both are reproduced in full in the appendix.

Appendix: shared test code

The test scripts above use the fixtures in conftest.py (browser setup, login, test user, cleanup) and the locators and actions in helpers.py. They are included here so every script in this document is complete.

tests/conftest.py

```
"""Shared pytest fixtures for the Selenium end-to-end tests.

The app under test is expected to be running already:
  - frontend (Vite)  -> http://localhost:5173
  - backend  (API)   -> http://localhost:3000

Command-line options:
  --browser=chrome|firefox   (default: chrome)
  --headless                 run without opening a browser window
  --base-url=URL             override the frontend URL
  --slow-mo=SECONDS          pause after every click, typing and page load
                             (handy for screen recordings), e.g. --slow-mo=1
"""

import json
import os
import time
import urllib.error
import urllib.request
from pathlib import Path

import pytest
from dotenv import load_dotenv
from selenium import webdriver
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.events import AbstractEventListener, EventFiringWebDriver
from selenium.webdriver.support.ui import WebDriverWait

import helpers

TESTS_DIR = Path(__file__).parent
load_dotenv(TESTS_DIR / ".env")

DEFAULT_BASE_URL = os.getenv("BASE_URL", "http://localhost:5173")
DEFAULT_TIMEOUT = 10
SCREENSHOT_DIR = TESTS_DIR / "screenshots"
TEST_IMAGE = TESTS_DIR / "fixtures" / "test-item.jpg"


def pytest_addoption(parser):
    parser.addoption(
        "--browser",
        action="store",
        default=os.getenv("BROWSER", "chrome"),
        choices=["chrome", "firefox"],
        help="Browser to run the tests in.",
    )
    parser.addoption(
        "--headless",
        action="store_true",
        default=os.getenv("HEADLESS", "").lower() in ("1", "true", "yes"),
        help="Run the browser in headless mode.",
    )
    parser.addoption(
        "--base-url",
        action="store",
```

```
        default=DEFAULT_BASE_URL,
        help="Base URL of the frontend under test.",
    )
    parser.addoption(
        "--slow-mo",
        action="store",
        type=float,
        default=float(os.getenv("SLOW_MO", "0")),
        help="Seconds to pause after each browser action (0 = full speed).",
    )


class _SlowMoListener(AbstractEventListener):
    """Pauses after each user-visible action so a human can follow along."""

    def __init__(self, delay):
        self.delay = delay

    def after_navigate_to(self, url, driver):
        time.sleep(self.delay)

    def after_click(self, element, driver):
        time.sleep(self.delay)

    def after_change_value_of(self, element, driver):
        time.sleep(self.delay)


def _build_driver(browser, headless):
    if browser == "firefox":
        options = webdriver.FirefoxOptions()
        if headless:
            options.add_argument("-headless")
        options.add_argument("--width=1400")
        options.add_argument("--height=900")
        return webdriver.Firefox(options=options)

    options = webdriver.ChromeOptions()
    if headless:
        options.add_argument("--headless=new")
    options.add_argument("--window-size=1400,900")
    options.add_argument("--disable-search-engine-choice-screen")
    return webdriver.Chrome(options=options)


def _register_user_via_api(user):
    request = urllib.request.Request(
        f"{helpers.api_url()}/auth/register",
        data=json.dumps(user).encode(),
        headers={"Content-Type": "application/json"},
        method="POST",
    )
    try:
        urllib.request.urlopen(request, timeout=10)
    except urllib.error.URLError as err:
        pytest.exit(
            f"Could not register a test user at {helpers.api_url()} - "
            f"is the backend running? ({err})"
        )


@pytest.fixture(scope="session")
def base_url(request):
    return request.config.getoption("--base-url").rstrip("/")
```

```
@pytest.fixture(scope="session")
def test_user():
    """Credentials for an account the tests can log in with.

    Uses TEST_USER_EMAIL / TEST_USER_PASSWORD from tests/.env if set, otherwise
    registers a brand-new user through the API once per test run, so tests
    never touch your real wardrobe.
    """
    email = os.getenv("TEST_USER_EMAIL")
    password = os.getenv("TEST_USER_PASSWORD")
    if email and password:
        return {"email": email, "password": password}

    stamp = int(time.time() * 1000)
    user = {
        "firstName": "Selenium",
        "lastName": "Tester",
        "phoneNumber": "",
        "email": f"selenium_{stamp}@example.com",
        "username": f"selenium_{stamp}",
        "password": "Password123!",
    }
    _register_user_via_api(user)
    return {"email": user["email"], "password": user["password"]}


@pytest.fixture
def driver(request):
    """A fresh browser per test, so cookies/auth never leak between tests."""
    drv = _build_driver(
        request.config.getoption("--browser"),
        request.config.getoption("--headless"),
    )
    slow_mo = request.config.getoption("--slow-mo")
    if slow_mo > 0:
        drv = EventFiringWebDriver(drv, _SlowMoListener(slow_mo))
    yield drv
    if slow_mo > 0:
        # Linger on the final screen so the end result is visible in a recording.
        time.sleep(slow_mo * 2)
    drv.quit()


@pytest.fixture
def wait(driver):
    """Explicit wait helper: wait.until(EC.visibility_of_element_located(...))."""
    return WebDriverWait(driver, DEFAULT_TIMEOUT)


@pytest.fixture
def open_page(driver, base_url):
    """Navigate to a path relative to the base URL, e.g. open_page("/login")."""

    def _open(path="/"):
        driver.get(f"{base_url}/{path.lstrip('/')}")
        return driver

    return _open


@pytest.fixture
def logged_in_driver(driver, wait, open_page, test_user):
    """A driver that has logged in through the UI and landed on /wardrobe."""
    open_page("/login")
    helpers.submit_login(driver, wait, test_user["email"], test_user["password"])
    wait.until(EC.url_contains("/wardrobe"))
```

```
    wait.until(EC.visibility_of_element_located(helpers.LOGOUT_BUTTON))
    return driver


@pytest.fixture
def test_image():
    """Absolute path to a small JPEG used when uploading clothing items."""
    return str(TEST_IMAGE.resolve())


@pytest.fixture
def unique_name():
    """A clothing item name that is different every run."""
    return f"Selenium Item {int(time.time() * 1000)}"


@pytest.fixture
def cleanup_items(logged_in_driver):
    """Collects names of items a test creates and deletes them afterwards.

    Usage: cleanup_items.append(name) right after creating an item.
    """
    names = []
    yield names
    for item in helpers.fetch_wardrobe_items(logged_in_driver):
        if item["name"] in names:
            helpers.delete_item_via_api(logged_in_driver, item["id"])


@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    """Save a screenshot whenever a test fails, to help debugging."""
    outcome = yield
    report = outcome.get_result()
    if report.when != "call" or not report.failed:
        return
    drv = item.funcargs.get("driver") or item.funcargs.get("logged_in_driver")
    if drv is None:
        return
    SCREENSHOT_DIR.mkdir(exist_ok=True)
    drv.save_screenshot(str(SCREENSHOT_DIR / f"{item.name}.png"))
```

tests/helpers.py

```
"""Locators and small helper functions shared by the tests."""

import os

from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import Select, WebDriverWait


def api_url():
    return os.getenv("API_URL", "http://localhost:3000/api")


# --- Locators ---------------------------------------------------------

LOGIN_EMAIL = (By.CSS_SELECTOR, "input[type='email']")
LOGIN_PASSWORD = (By.CSS_SELECTOR, "input[type='password']")
LOGIN_SUBMIT = (By.CSS_SELECTOR, "button[type='submit']")
LOGIN_ERROR = (By.CSS_SELECTOR, "form p.text-red-600")

LOGOUT_BUTTON = (By.XPATH, "//header//button[normalize-space()='Logout']")
```

```
ADD_ITEM_BUTTON = (By.XPATH, "//li[normalize-space()='Add']")
ADD_ITEM_FORM_TITLE = (By.XPATH, "//form//h2[normalize-space()='Add Clothing Item']")
NAME_INPUT = (By.ID, "name-input")
BRAND_INPUT = (By.ID, "brand-input")
TYPE_SELECT = (By.ID, "type-input")
SIZE_INPUT = (By.ID, "size-input")
COLOR_INPUT = (By.ID, "color-input")
IMAGE_INPUT = (By.ID, "image-input")
DESCRIPTION_INPUT = (By.ID, "description-input")
ADD_ITEM_SUBMIT = (By.XPATH, "//form//button[normalize-space()='Add Item']")

# Uploads go to Cloudinary, so allow longer than the default wait.
UPLOAD_TIMEOUT = 30


def item_card_link(name):
    """The clickable card for a clothing item in the wardrobe grid."""
    return (By.CSS_SELECTOR, f"a[aria-label='View details for {name}']")


def item_heading(name):
    """The item name heading on a clothing item's detail page."""
    return (By.XPATH, f"//h1[normalize-space()='{name}']")


def button_with_text(text):
    return (By.XPATH, f"//button[normalize-space()='{text}']")


# --- Actions ------------------------------------------------------------


def submit_login(driver, wait, email, password):
    wait.until(EC.visibility_of_element_located(LOGIN_EMAIL)).send_keys(email)
    driver.find_element(*LOGIN_PASSWORD).send_keys(password)
    driver.find_element(*LOGIN_SUBMIT).click()


def open_add_item_form(driver, wait):
    wait.until(EC.element_to_be_clickable(ADD_ITEM_BUTTON)).click()
    wait.until(EC.visibility_of_element_located(ADD_ITEM_FORM_TITLE))


def fill_add_item_form(
    driver, name=None, brand=None, item_type=None, size=None, color=None, image_path=None
):
    """Fill in whichever fields are given; leave the rest empty."""
    if name:
        driver.find_element(*NAME_INPUT).send_keys(name)
    if brand:
        driver.find_element(*BRAND_INPUT).send_keys(brand)
    if item_type:
        Select(driver.find_element(*TYPE_SELECT)).select_by_visible_text(item_type)
    if size:
        driver.find_element(*SIZE_INPUT).send_keys(size)
    if color:
        driver.find_element(*COLOR_INPUT).send_keys(color)
    if image_path:
        driver.find_element(*IMAGE_INPUT).send_keys(image_path)


def add_item(
    driver, wait, name, image_path, brand="Selenium", item_type="Tops", size="M", color="Navy"
):
    """Add an item through the UI and wait until its card shows in the wardrobe."""
```

```
    open_add_item_form(driver, wait)
    fill_add_item_form(
        driver,
        name=name,
        brand=brand,
        item_type=item_type,
        size=size,
        color=color,
        image_path=image_path,
    )
    driver.find_element(*ADD_ITEM_SUBMIT).click()
    # The form uploads the image, then reloads the page.
    return WebDriverWait(driver, UPLOAD_TIMEOUT).until(
        EC.visibility_of_element_located(item_card_link(name))
    )


# --- Direct API checks (run inside the logged-in browser) ------------------
# These use the browser's own login cookie, so they see exactly what the
# signed-in user would. Useful for confirming something is really gone
# rather than just not rendered yet.


def fetch_wardrobe_items(driver):
    return driver.execute_async_script(
        """
        const done = arguments[arguments.length - 1];
        fetch(arguments[0] + "/clothing/get", { credentials: "include" })
          .then((res) => (res.ok ? res.json() : []))
          .then(done)
          .catch(() => done([]));
        """,
        api_url(),
    )


def delete_item_via_api(driver, item_id):
    driver.execute_async_script(
        """
        const done = arguments[arguments.length - 1];
        fetch(arguments[0] + "/clothing/" + arguments[1], {
          method: "DELETE",
          credentials: "include",
        }).finally(() => done());
        """,
        api_url(),
        item_id,
    )
```